PolyglotTalk: A Context-Aware Cascade Speech-to-Speech Translation Pipeline for Low-Resource Offline Deployment

Dr.Sayeekumar M
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
(sayeekumar.m@jainuniversity.ac.in)

Panshul Pratik Sarma
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
pans83372@gmail.com

Dhritiraj Lahakar
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
dlahkar69@gmail.com

Yash Karecha
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
yash.karecha@outlook.com

Shrijith S Menon
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
shrijithsmenon@gmail.com

Soumyajit Chanda
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
s.chanda.wrk@gmail.com

***Abstract*—**Real-time multilingual speech communication remains inaccessible on low-end, offline devices. Existing end-to-end Speech-to-Speech Translation (S2ST) systems—such as Google's 2025 end-to-end model—require server-grade hardware and cloud connectivity, leaving a deployment gap for resource- constrained scenarios. We present PolyglotTalk, a cascade S2ST prototype consisting of a 4-thread pipeline: faster-whisper (ASR)

→ Argos Translate (MT) → Facebook MMS-TTS (TTS). The primary technical contribution is a rolling context window mechanism—a deque-based prefix injection strategy applied during machine translation—which prevents repetition and grammar breaks across chunk boundaries in streaming translation. Benchmarking each pipeline component independently (WER, BLEU, per-stage latency) and evaluating the full end-to-end system. Results demonstrate sub-3s end-to-end latency on CPU hardware, with the context window reducing translation repetitions from 4– 9 instances to 0–1 across 10 consecutive sentences. This paper presents the architecture, experimental results, and a foundation for future mobile calling integration.

Index Terms—speech-to-speech translation, cascade architecture, context continuity, low-resource deployment, offline translation

I. Introduction

India has 22 officially recognized languages and over 19,500 dialects. This creates a constant communication barrier between linguistic communities. While more urban populations are using English as a common language, rural and semi-urban areas, which make up over 65% of India's 1.4 billion people, are still isolated linguistically. Real-time speech-to-speech translation (S2ST) may offer a solution to bridge the gap, providing seamless multilingual communication in healthcare, education, business, and government.

Recent advances in end-to-end (E2E) S2ST have demonstrated promising results in quality and latency. Google's announcement in November 2025 about a real-time E2E speech translation model marks an important achievement. It reaches nearly human-level translation quality with sub-second latency. Research systems like Translatotron, Translatotron 2, and Meta's Seamless have pushed the limits of direct speech-to-speech modeling. However, these systems have a major drawback: they need a lot of computing power, usually server-grade GPUs with 16 to 40 GB VRAM, and constant cloud access. This reliance makes them unworkable in situations with limited resources, such as low-end smartphones, rural areas with spotty connectivity, and sensitive applications where data can't leave the device.

The difference between state-of-the-art E2E systems and constrained conditions is what motivates us in our work. We present a cascade S2ST system dubbed PolyglotTalk that was developed with the purpose of being executed offline on ordinary machines. In contrast with other E2E systems that contain numerous stages, PolyglotTalk separates the process of translation into three steps, namely ASR, MT and TTS in a four-thread pipeline. The tradeoff here is the sacrifice of elegance for usability. This allows each of the modules to be independently trained.

Technical Contribution: Context-Aware Streaming:

The core technological innovation of this research is a rolling context window scheme, designed to solve the major challenge that lies at the heart of S2ST cascading translation-how to guarantee the cohesion of semantics and grammar between chunk boundaries. In streaming real-time scenarios, audio input is chunked into relatively short intervals of 3 to 5 seconds for latency reduction purposes. Yet, translation of individual segments without taking the previous context into account causes the following issues:

Repetition: Since the machine translation engine may be inclined to reproduce previously heard fragments, it may result in repetition of output (e.g., "Hello, how are you? Hello, how are you doing?")

Grammar breaks: There is a lack of cohesion of pronouns, verb tenses, and discourse markers (e.g., He went to the store" followed by "She bought milk" when actually referring to one and the same person).

Semantic shift: In lack of context, the meaning of ambiguous terms becomes distorted ("bank" as financial institution versus river bank).

The proposed approach utilizes a prefix injection technique based on a deque structure that applies in the MT stage. It adds the translations of previous N chunks (N=2 in our case) to the incoming ASR segment, orders MT to translate the entire concatenated string, and retains only new parts.

A. System Overview

PolyglotTalk consists of a multi-layered architecture comprising four core components operating as concurrent threads:

1) **Audio Capture Thread**: Captures microphone input as overlapping 2.5-second audio chunks (stride 1.5s, overlap 1.0s) using the sounddevice library, and places each chunk on a thread-safe queue for downstream processing.
2) **ASR Thread**: Transcribes audio chunks using a faster version of whisper [9], a CTranslate2-based reimplementation of OpenAI's Whisper model [10] that reduces memory usage and inference latency through INT8 quantisation. To handle boundary artefacts introduced by overlapping chunks, the thread applies a normalised suffix/prefix deduplication pass before forwarding transcription output.
3) **Translation Thread**: Translates ASR output using Argos Translate [11], which is an offline neural multi-model neural machine translation system built on top of OpenNMT-py framework and OPUS language corpora [12]. The thread applies the concept of the rolling context window by
4) prepending previously translated source sentence pieces to the input.
5) **TTS Thread**: Text-to-speech conversion of the translated text is performed using Meta's MMS TTS models[8], which work for 1,100+ languages. Synthesized audio is written to WAV files rather than played through a speaker, preventing microphone feedback.

Inter-thread communication is done through thread-safe queues, allowing asynchronous execution in low latency performance. The entire application operates offline on a consumer laptop (Intel i5-8250U, 8GB).

B. Evaluation and Results

Our prototype is evaluated using four sets of experiments:

1) **ASR Benchmarking**: Faster-whisper vs Whisper on LibriSpeech test-clean[13], evaluating Word Error Rate (WER) and latency.
2) **MT Benchmarking**: Argos Translate vs MarianMT on a 25-sentence English-to-Hindi test, evaluating BLEU scores[14] and latency.
3) **End-to-End Latency**: Evaluating latency for each stage, as well as overall.
4) **Context Continuity Ablation**: Quantifying repetitions and grammatical errors in translation both with and without the context window, for ten sentences at once.

PolyglotTalk proves to be able to achieve an under 3s end-to-end latency on CPU devices. With the context window, translation repetitions are reduced from 4-9 to 0-1 for each sentence in the test set. Even if absolute translation quality (BLEU≈ 15-18)is still lower than in commercial solutions, we demonstrate the feasibility of our prototype.

C. Scope and Limitations

This paper demonstrates a **proof-of-concept system**, not a deployable system. We consider only the English-to-Hindi language pair and perform evaluations using synthetic data, not real conversation exchanges. Important limitations of our approach include:

- **Poor Translation**: BLEU scores produced by Argos Translate (15-18) are much lower than those generated by cloud-based solutions (30-40+).
- **TTS Speed limitation**: The synthesis delay associated with MMS-TTS (1.2-1.8s per sentence) represents the largest source of latency in our end-to-end system.
- **No Voice Activity Detection (VAD)**: Our system processes 3-second audio chunks, resulting in mid-word cuts and artificial silences
- **Limited to one Language Pair**: Expanding support to India's 22 official languages would require downloading new models and implementing routing logic.

PolyglotTalk successfully showcases cascade S2ST with contextualized streaming capabilities on constrained hardware, laying groundwork for mobile deployment.

D. Paper Organization

The rest of this paper is structured as follows Section II provides background literature on cascade and end-to-end S2ST, streaming translation, and context continuity.Section III describes system design and the rolling context window approach. Section IV discusses experimental setup, datasets, and evaluation criteria. Section V provides baseline results. Section VI provides results discussion. Section VII covers future directions in VAD integration, multilingual routing, and models for indic languages.

II. Related Work

A. Cascade Speech-to-Speech Translation

The Cascade S2ST paradigm involves splitting the translation process into three consecutive steps: ASR, MT, and TTS. This system has remained a popular approach for many years because of its flexibility, interpretability, and scalability.

Each component of the Cascade architecture can be optimized and improved individually and does not require training the entire pipeline. Unfortunately, cascaded S2ST systems are prone to error propagation, where ASR mistakes lead to MT misinterpretation, and poor prosody is generated by the speech-to-text synthesis due to unnatural MT output.Recently, many researchers have been working to optimize existing S2ST cascade components to provide them in low-resource applications.

The faster-whisper library [9] has utilized CTranslate2 quantization technique to make OpenAI's Whisper model [10] four times faster while reducing memory usage by 50% with

minor WER changes. For translation, the Argos Translate service [11] provides offline neural translation by incorporating OpenNMT models that are trained using OPUS parallel corpora [12].In order to implement TTS, Facebook's MMS framework [8] scales the technology to cover more than 1,100 languages using unsupervised learning methods based on unlabeled speech data.

Despite many improvements made to the cascaded S2ST architecture, none of the previous works have considered context continuity for streaming applications. Currently, all streaming implementations use a window of audio input to generate translation output, which results in repetitions and grammar inconsistencies.

B. *End-to-End Speech-to-Speech Translation*

End-to-end S2ST systems directly map source speech to target speech without intermediate text representations, potentially avoiding error propagation and enabling prosody transfer. Translatotron [2] pioneered this approach with a sequence-to-sequence model that generates target spectrograms from source audio. Translatotron 2 [3] improved quality and added voice preservation by conditioning on speaker embeddings. UnitY [7] introduced a two-pass architecture using discrete speech units, achieving state-of-the-art quality on multilingual benchmarks.

Recent systems have focused on streaming and low-latency deployment. SimulTron [4] demonstrates on-device simultaneous S2ST using model compression and streaming decoding, achieving sub-2s latency on mobile GPUs. DASpeech [5] applies directed acyclic transformers to reduce autoregressive decoding overhead. Meta's Seamless [6] combines stream- ing ASR, MT, and TTS with expressive prosody modeling, supporting 100+ languages with real-time latency on server hardware.

Google's November 2025 announcement [1] represents the current state-of-the-art, claiming near-human translation quality with sub-second latency. However, technical details remain unpublished, and the system requires cloud connectivity. While E2E systems achieve superior quality and latency on high-end hardware, their monolithic architecture and resource requirements limit deployability in offline, low-resource scenarios— the target use case for PolyglotTalk.

C. *Streaming Translation and Simultaneous Interpretation*

Streaming translation—translating speech in real-time as it is spoken—poses unique challenges beyond offline batch translation. The system should balance latency (translation speed) with quality, including both accuracy and fluency. Research in this area has been spurred by the IWSLT Simultaneous Translation Shared Task [16], which has set a standard for the relationship between latency and quality. Popular methods include:

- Wait-k algorithms [17]: The translator waits for k source tokens before starting translation, sacrificing latency for contextual knowledge.
- Prefix-to-prefix architecture: The network is trained to translate parts of the source into parts of the target, allowing the model to incrementally decode the source sequence.
- **Adaptive policies:** The model decides when to wait for additional information and when to produce a translation using a confidence score.

Most of these approaches are designed for end-to-end models and rely on the differentiability of jointly trained architectures. Applying them to cascade models, where ASR, MT, and TTS are trained separately, is an unsolved problem. Rolling window context allows for a more straightforward, trainable-free solution that works for the MT component alone without changing any underlying architecture.

D. *Context Continuity in Machine Translation*

Document-level MT has always considered the need to incorporate document-level context, such as previous sentences, into the process. Early research used cache models to favor translations that contain words that were already seen earlier. Neural MT models have experimented with models that either jointly encode several sentences or condition their predictions on document-level representations. Yet, these methods require the model to be aware of the entire document or have access to extended context windows during training.

In the case of S2ST, there is also an incremental arrival of context, as well as arbitrary-length dialogues, which cannot be processed by an increasing amount of memory allocation. The deque-based solution for context modeling allows for maintaining a sliding window of a fixed size of the latest translations, providing enough context for avoiding repetition and grammar inconsistency but without having growing memory usage.

The SpeechMatrix project [18] proved that speech-to-speech corpora may be used to improve context modeling in E2E systems; however, large amounts of parallel speech data are needed, which are unavailable in many language pairs. The suggested method works in the textual level during the cascade processing, utilizing context abilities of current MT models without additional training.

III. System Architecture

This paper introduces a four-thread pipeline structure tailored for efficient offline S2ST in real-time operation on CPU. System structure is illustrated below. All threads work independently and communicate through thread-safe queues to reduce blocking and increase efficiency. Herein, the description of all components, including the context window process, is provided

A. *Pipeline Overview*

Audio data is captured by the system in 3 seconds increments by the following sequence of actions:

1. - Audio Capturing: captures the audio from the microphone and puts raw PCM data to the queue.
2. - ASR: transcribes audio fragments to the source language text.
3. - MT with Context Window: translates source text into the target language text while preserving the context.
4. - TTS: generates speech by synthesizing the translated text.

All tasks run in independent threads, allowing us to create a pipeline, where TTS synthesizes fragment n while MT performs translation of fragment n+1, ASR transcribes fragment n+2, and the first audio thread captures fragment n+3. It helps to reduce the latency time comparing to sequential algorithm operation.

B. *Audio Capture Module*

The audio capture thread uses PyAudio to record 16 kHz mono audio in 3-second chunks. Audio is stored as 16-bit PCM and

enqueued for ASR processing.The 3-second duration was chosen as a compromise between lower latency (shorter chunks produce less delay) and higher ASR precision (as more context improves recognition accuracy). Limitation: Currently, no Voice Activity Detection (VAD) is implemented; audio chunks are obtained with fixed periods, regardless of the voice activity. Thus, two problems arise: (1) cutting the words that start at one chunk and end at another; and (2) unnecessary processing of silent chunks. We plan to include VAD in future research in order to achieve natural audio chunk segmentation.

C. ASR Module

Audio capture uses PyAudio library for capturing 16 kHz mono audio in 3-second intervals. Audio samples are saved as 16-bit PCM and passed automatically for ASR processing. The selection of 3 seconds as chunk length is based on latency/ASR accuracy trade-off where shorter lengths reduce latency but provide less contextual information for acoustic models. The chunks are recorded at fixed time intervals independent of speech activity, resulting in two problems: (1) speech truncation in the middle of a word and (2) wasteful processing of silent periods. Further work will add Voice Activity Detection (VAD) for more natural segmentation.

The ASR process utilizes faster-whisper [9] based on CTranslate2 optimization of the OpenAI's Whisper model [10]. Whisper is a transformer-encoder-decoder ASR model trained on 680,000 hours of multilingual speech that provides high performance in terms of accent diversity, background noise, and domains. Faster-whisper adds three improvements over the original: quantization to 8-bit reduces model size by half to 1.5 GB with only <1% increase in word error rate (WER), CTranslate2 inference provides optimized CPU/GPU kernels and speedup by ~4 compared to PyTorch inference, and batching allows to perform parallel processing on multiple chunks if possible.

In order to obtain accurate results for ASR of English language, the model base.en with 74 M parameters is selected. Its performance on LibriSpeech dataset test-clean is 3.2% WER with a mean of 0.8 seconds delay on current hardware (Section V-A). Other configuration parameters include beam_size = 5, best_of = 5, temperature = 0.0 for deterministic decoding, and language = 'en' to exclude language detection delays.

D. Machine Translation (MT) Algorithm with Context Window Mechanism

The machine translation (MT) part of our algorithm represents the main technical innovation introduced by this project. It consists of rolling context windows that preserve both semantics and grammar between the translated ASR chunks.

Baseline Algorithm (Context-Free): A very basic solution for a streaming machine translation system translates each ASR chunk individually:

```
for chunk in asr_output:
    translation = mt_model.translate(chunk)
    tts_queue.put(translation)
```

This baseline algorithm suffers from the following three problems:

1. **Repetitive translation**: The machine translator is unaware of previous translations and can translate already known phrases within consecutive ASR chunks. For example, the phrase "Hello, how are you?" followed by "I am fine" will be translated as "Namaste, aap kaise hain?" (Hello, how are you?), and then "Na-maste, main achha hoon" (Hello, I am well), thus repeating itself twice..
2. **Grammar inconsistency**: Discourse markers, pronouns, verbs, etc., can break down and become inconsistent. Translating "He went to the store" then "Bought some milk" would result in a grammatically wrong phrase since the second chunk does not have any discourse marker that indicates its subject.
3. **Semantic ambiguity**: Some words may be ambiguous when used out of context and should therefore be translated accordingly. For example, when translating "I went to the bank," the MT algorithm must understand whether it is a place to store money or it is a natural bank by taking context into account.

Our solution uses the sliding window that contains the N most recent translations (N=2 in our case). For each ASR chunk, the system:

- Adds the last N translations as the context.
- Translates the concatenation of context and the current ASR chunk.
- Removes everything but the current ASR chunk's translation.
- Puts back the result back into the sliding window queue, which removes the old translation if the queue's length is greater than N.

Algorithm 1: Rolling Context Window for Streaming MT

1. **Input:** ASR chunk stream $\{c_1, c_2, \ldots\}$
2. **Output:** Translation stream $\{t_1, t_2, \ldots\}$
3. Initialize the context deque D to the empty set (maximum capacity N = 2).
4. for each chunk ci derived from the ASR output do
5. prefix ← concatenate(D)
6. full _ text ← prefix + c_i
7. full _ translation ← MT(full _ text)
8. *ti ← extractnewportion(fulltranslation, prefix)*
9. D.append(t_i)
10. **if** $|D| > N$ **then**
11. D.popleft()
12. **end if**
13. Enqueue t_i to TTS
14. **end for**

Key Design Decisions:

- Context size N = 2: Based on empirical findings, using two previous translations is enough context to ensure proper translation and avoid repetitions and grammar mistakes. Further increases in N would result in higher delays for MT without corresponding increases in quality.
- Text-level context: Translated, not source text is stored, since this will cause the MT model to process its own previous outputs,ensuring consistent terminology usage and coherent style.
- Extraction heuristic: The new portion is calculated by finding

the length of the prefix translation and cutting off all the preceding text from that point in the full translation text. The assumption that this technique will work is made based on the idea that the MT model will translate the prefix text identically no matter whether it is a part of a longer phrase or standalone.

E. MT Model:

Argos Translate: For the used neural MT engine we opted for Argos Translate [11] based on the OpenNMT framework and trained on OPUS [12]. Argos Translate offers pre-trained models for more than 40 languages pairs and can be fully offline, as it requires neither API calls nor any Internet connection whatsoever. For our purposes, we use the English-Hindi transformer model with 6 layers and 60 millions of parameters, trained on 1.2 million sentence pairs. Although Argos' BLEU score (varying from 15 to 18 for our test dataset mentioned in section V-B) lags behind other systems, the model runs exclusively on the CPU, producing results in 0.3-0.5 seconds.latency per sentence, This arrangement makes it possible for the model to be streamed in real time. Alternate approach evaluated: Initially, a BLEU score comparison between MarianMT (from Hugging Face's Transformers package) resulted in higher scores of about 22-25; but the long processing time of 1.2-1.5 seconds per sentence made it unacceptable for use in real time.

F. TTS Module

The TTS thread uses Facebook's MMS-TTS models [8], which support 1,100+ languages including all 22 official Indian languages. MMS-TTS is based on VITS (Variational Inference with adversarial learning for end-to-end Text-to- Speech), a neural vocoder that generates waveforms directly from text.

The Hindi TTS model (200M parameters) is trained on 10 hours of read speech data. Synthesis quality is intelligible but exhibits robotic prosody and occasional mispronunciations of English loanwords. Synthesis latency averages 1.2–1.8s per sentence on CPU. As such, TTS is currently the primary factor in our pipeline's latency (Section V-C).

Audio Playback: Synthesized audio is played back using PyAudio.The system uses a dedicated playback thread to allow continuous operation of the TTS thread.

G. Thread Synchronization and Error Handling

Threads communicate via queue.Queue objects provided by Python's threading module, which guarantees FIFO semantics of the communication channel. Each queue has maximum size set to 10, in order to prevent uncontrolled memory growth if the downstream worker threads fall behind.

Error handling: Each worker thread catches exceptions during the processing loop (such as ASR timeout, MT failure, or TTS crash) and logs the error together with the corresponding chunk. The process resumes from the next chunk.

Graceful shutdown: A global stop_event variable can be accessed by any thread and allows the main thread to terminate all other threads when needed. Workers drain their queues upon receiving stop signal in order not to lose in-flight data.

IV. Experimental Setup

The evaluation of PolyglotTalk is carried out using four experiments meant to test both individual components and the entire end-to-end system. Below is the description of the hardware, data, metrics, and experimental settings. datasets, metrics, and experimental procedures.

A. Hardware and Software Environment

All experiments run on a consumer-grade laptop with the following specifications:

- **CPU**: Intel Core i5-8250U (4 cores, 8 threads, 1.6 GHz base / 3.4 GHz turbo)
- **RAM**: 8 GB DDR4
- **Storage**: 256 GB SSD
- **OS**: Ubuntu 22.04 LTS
- **Python**: 3.11.9

No GPU acceleration was used during the experiments. The hardware above is representative of an entry-level laptop, or even a mid-range smartphone, considering the difference in performance between ARM and x86.

Software Dependencies:

- faster-whisper 1.1.0 (Whisper base.en model)
- transformers 4.49.0 (MMS-TTS model loading)
- torch 2.10.0
- argostranslate 1.9.6 (English-Hindi model package)
- sounddevice 0.5.1 (Microphone audio capture)
- soundfile 0.13.1 (WAV file output)
- Numpy 1.26.4 (Audio buffer processing)
- sacrebleu 2.6.0 (for BLEU scoring)
- jiwer 4.0.0 (for WER calculation)

B. Datasets

1) Evaluation of ASR Performance: LibriSpeech: To evaluate ASR performance, we use the LibriSpeech test-clean dataset [13]. It is one of the standard ASR corpora, which is a public-domain audio-book corpus, consisting of 5.4 hours of speech with 2,620 utterances. This clean test set offers a high-quality signal with little ambient noise for measuring word error rates (WERs)..

Preprocessing: All audio samples are resampled at 16 kHz with a single-channel configuration to ensure compatibility with the ASR model's input expectations. Further processing steps (e.g., noise reduction, normalization) have not been taken.

2) Evaluation of MT: Custom English-Hindi Test Set: For evaluating machine translation performance, we develop a custom English-Hindi parallel text of 25 sentences based on three domains:

a.Conversational (10 sentences): Greetings, chitchat, and frequently used expressions (e.g., "How do you feel?", "What's your name?").

b.Transactional (10 sentences): Shopping, directions, and requests for service (e.g., "How much is that?", "Where is the nearest hospital?").

c. Technical (5 sentences): Simple technical instructions (e.g., "Restart your computer please", "Press submit button"). Translations from English into Hindi are made by an English-Hindi translation specialist, then checked by a native Hindi speaker. Sentence lengths are between 5 and 15 words (average of 9.2 words) to simulate typical spoken sentences.

Reasoning: In developing the test set, we opted to create a custom data collection as opposed to using MT benchmark tests (WMT, FLORES) for the following reasons:

(1) Benchmarks are mostly designed for evaluating written text, while S2ST involves spoken language with simpler syntax/vocabulary;

(2) Our test set includes domain-specific phrases related to the specific application area (multilingual interaction in India).

3) Context Continuity Evaluation: 10-Sentence Sequence: Context continuity evaluation requires a test set with at least 10 English sentences to detect repetitions and grammatical

a. "Hello, how are you?"
b. "I am doing well, thank you."
c. "What are you doing today?"
d. "I am going to the market."
e. "That sounds interesting."
f. "Yes, I need to buy some vegetables."
g. "Which vegetables do you need?"
h. "I need tomatoes and onions."
i. "Those are very common ingredients."
j. "Yes, I use them in many dishes."

This sequence contains:

- **Pronouns** ("I", "you") require consistent reference.
- **Discourse markers** ("Yes", "That") linking to prior context.
- **Anaphora** ("Those" referring to "tomatoes and onions"). We translate this sequence with and without the context window, manually counting repetitions and grammar errors in the Hindi output.

C. *Evaluation Metrics*

1) Word Error Rate (WER): WER measures ASR accuracy as the edit distance between the hypothesis (ASR output) and reference (ground truth) transcripts, normalized by the number of words in the reference:

$$\text{WER} = \frac{\text{Substitutions + Insertions + Deletions}}{\text{Total Words in Reference}}$$

Computation of WER using the jiwer library is done[15], which implements the standard Levenshtein distance algorithm with case-insensitive matching and punctuation normalization.

2) BLEU Score: BLEU (Bilingual Evaluation Understudy) measures MT quality by comparing *n*-gram overlap between the hypothesis (MT output) and reference translations. We use sacrebleu [14] to compute sentence-level and corpus- level BLEU scores with the default configuration (BLEU-4, smoothing method 1).

Interpretation: BLEU scores range from 0 (no overlap) to 100 (perfect match). Scores above 30 are generally considered acceptable for commercial systems; scores below 20 indicate significant quality issues.

3) Latency: ASR, MT, TTS latencies and end-to-end latency (audio to audio processing) are expressed in seconds. Latency numbers are obtained by taking an average of 10 trials to mitigate variance due to CPU scheduling and I/O.

End-to-End Latency: Time taken from recording of 3-second audio clip until start of translation audio play. This includes ASR, MT, TTS, and queuing times.

4) Context Continuity Metrics (for 10 sentence context test):

1) Repetition: Number of repeated phrases/words used in the previous sentences like greeting and subject words.

2) Grammar Errors: Number of sentences with inconsistency in pronouns and verb tenses and lack of discourse markers.

Two native Hindi speakers are manually scored for their output; if there is a difference in scores, a discussion will be held. Manual scoring is needed since BLEU/METEOR scores are insensitive in scoring context continuity.

D. *Experimental Procedures*

1. ASR Benchmarking: **Objective**: Compare faster-whisper with standard Whisper (PyTorch implementation) regarding the Word Error Rate (WER) and latency.

Procedure:

I. Load LibriSpeech test-clean (2,620 utterances).
II. Transcribe each utterance using:
   - faster-whisper (base.en, 8-bit quantized)
   - Whisper (base.en, PyTorch, FP32)
III. Calculate WER and average latency per utterance.

2. MT Benchmarking: **Objective**: Compare Argos Translate with MarianMT regarding BLEU scores and latency.

Procedure:

I. Load the test set containing 25 sentences in English and Hindi languages.
II. Translate each sentence using:
   - Argos Translate (English-Hindi model)
   - MarianMT (Helsinki-NLP/opus-mt-en-hi)
III. Calculate sentence-level and corpus-level BLEU scores.
IV. Measure average latency per sentence.

3. End-to-End Latency: **Objective**: Calculate latencies per-stage and for the entire pipeline.

Procedure:

I. Capture 10 English utterances (3 seconds each) using the audio capture module.
II. Process each utterance through the full pipeline (ASR → MT → TTS).
III. Record timestamps at each stage: audio capture complete, ASR complete, MT complete, TTS complete, audio playback start.
IV. Calculate per stage latency as the difference between two subsequent timestamps as well as total latency from audio capture beginning till audio playback beginning.

4. Context Continuity Ablation: **Objective**: Evaluate the effect of rolling context window on repetitions and grammar mistakes.

Procedure:

I. Translate the 10-sentence sequence using:
   - **Baseline**: No context (each sentence translated independently).
   - **Context Window**: Rolling context with $N = 2$ prior translations.
II. Manually count repetitions and grammar errors in each output.
III. Compare results for both method.

V. Results

This section includes benchmark results for each experiment. Unless specified otherwise, all the data provided are average results over 10 executions.

A. *ASR Benchmarking*

Table I provides a performance benchmark of faster-whisper against standard Whisper on the LibriSpeech test-clean set.

Key Findings:

- faster-whisper is **4× faster** (0.8s vs. 3.2s per 3-second chunk) with only 0.1% additional WER.
- Both models have WER < 3.5%, well below the 5% threshold for acceptable ASR quality.
- 0.8 s of latency makes faster-whisper feasible for real-time streaming applications, whereas 3.2 s of latency would introduce an uncomfortable delay.

TABLE I
ASR PERFORMANCE ON LIBRISPEECH TEST-CLEAN (2,620 UTTERANCES)

Model	WER (%)	Latency (s)
Whisper (base.en, PyTorch)	3.1	3.2
faster-whisper (base.en, 8-bit)	3.2	0.8

B. MT Benchmarking

Table II provides a performance benchmark of Argos Translate against MarianMT on the 25-sentence English-Hindi test set.

TABLE II
MT PERFORMANCE ON ENGLISH-HINDI TEST SET (25 SENTENCES)

Model	BLEU	Latency (s)
Argos Translate	15.3	0.4
MarianMT (opus-mt-en-hi)	22.7	1.3

Key Findings:

- MarianMT is **48% more precise** (BLEU 22.7 vs. 15.3) but at **3.25× larger latency** (1.3s vs. 0.4s).
- Argos Translate has a BLEU score of 15.3, which is below the 20-point threshold, meaning that the system translates text inaccurately.
- Still, Argos Translate has 0.4 s of latency which makes it eligible for real-time streaming. MarianMT's 1.3 s of latency would make the system non-responsive.
- Argos Translate was chosen as the default MT engine as latency is prioritized over BLEU for this application. Section VI-B discusses this trade-off in detail.

C. End-to-End Latency

Table III demonstrates the breakdown of latencies at every stage, as well as the end-to-end latency.

TABLE III
END-TO-END LATENCY BREAKDOWN (10 UTTERANCES, 3S EACH)

Stage	Latency (s)
Audio Capture	3.0 (fixed) ASR
(faster-whisper)	0.8
MT (Argos Translate)	0.4
TTS (MMS-TTS)	1.5
Queue Waiting	0.2
Total End-to-End	**2.9**

Key Findings:

- **TTS is the bottleneck**: TTS serves as the most resource-intensive component, requiring 1.5 s and accounting for 52% of the processing latency (except for the fixed 3 s for audio capture).
- Total latency is equal to 2.9 s; thus, users receive the translated audio 2.9 seconds after they finish speaking (with another fixed 3 s before).
- ASR and MT stages combined consume only 1.2 s (which accounts for 41% of the latency), demonstrating the efficiency of faster-whisper and Argos
- Queue latency takes 0.2 seconds which is neglible; hence, a 4-thread pipeline ensures optimal overlapping of computations.

D. Context Continuity Ablation

Table IV shows how the rolling context window affects repetition and grammar errors.

TABLE III
END-TO-END LATENCY BREAKDOWN (10 UTTERANCES, 3S EACH)

Condition	Repetitions	Grammar Errors
Baseline (No Context)	7	4
Rolling Context Window ($N = 2$)	1	0

Key Findings:

● The baseline model generates 7 repetitions in 10 sentences. Examples of common repetitions are:

○ Repetitive translations of greeting words (e.g., Namaste/ Hello occurs 3 times).

○ Redundant inclusion of subject words (e.g., “main”/I).

○ Unnecessary inclusion of discourse markers (e.g., haan/ Yes).

● There are 4 grammar errors in the baseline model. These errors can be categorized as:

○ Incorrect pronouns (e.g., using he/she interchangeably).

○ Verb tense agreement (past and present tenses).

○ Anaphora error (“those” without its antecedent).

● Using a rolling context window reduces repetitions to 1 (86% reduction) and all grammar errors are eliminated.

● Sentence 9 has one repetition as a result of limited context size (N=2). Including one phrase from sentence 6 solves this problem. However, N must be set to 3, which results in increased MT latency.

E. TTS Quality Evolution

Not being a part of the experiment itself, qualitative analysis reveals a certain enhancement in the quality of TTS engines during development. Three of such engines have been examined belo

TABLE V
TTS ENGINE COMPARISON (QUALITATIVE)

Engine	Quality	Latency (s)	Status
gTTS (Google)	High	2.5	Requires internet
pyttsx3	Low	0.5	Robotic, unintelligible
MMS-TTS	Medium	1.5	Offline, acceptable

Key Findings:

- **The gTTS** (Google Text-to-Speech) yields the best results in terms of audio quality but requires an internet connection. This violates our offline principle.

- **The pyttsx3** (offline, rule-based engine) works quickly (up to 0.5 second) and provides robotic, incomprehensible voice;

- **The MMS-TTS** offers a balanced quality-latency compromise. The prosodic performance is comprehensible but not as natural as in gTTS. The latency of 1.5 seconds is higher than the desired value but still acceptable. In future experiments,

we will consider other engines such as the AI4Bharat's IndicF5 [20] to alleviate the TTS bottleneck.

VI. Discussion

In the current section, we proceed to discuss the obtained results, outline possible drawbacks of the adopted system architecture and its advantages and limitations.

A. Viability of Cascade vs. End-to-End Architectures

Based on our experimental results, cascade S2ST is feasible in low-resource offline environments regardless of the superiority of E2E architectures theoretically. This is mainly because **modularity enables aggressive fine-tuning** the different parts separately without re-training:

faster-whisper reaches 4× higher efficiency compared to standard Whisper thanks to quantization and optimized inference with almost no impact on Word Error Rate (WER) (3.2% vs. 3.1%).

Argos Translate allows offline machine translation with a latency of 0.4 seconds allowing real-time streaming despite worse BLEU scores.

MMS-TTS extends the coverage up to more than 1,100 languages, including all 22 official languages in India, with no need for language-specific training data..

On the other hand, the end-to-end approach employed in Translatotron [2], Seamless [6], and Google's 2025 model [1] requires re-training the entire pipeline to improve latency or add another supported language. Though the results from E2E systems are more accurate with optimal hardware performance, their monolithic nature hinders flexibility.

Error Propagation: Error propagation between consecutive modules is frequently cited as the key issue of cascade architecture.More specifically, any mistakes made during the ASR are amplified by the MT engine, while unnatural MT leads to prosodically inferior results in TTS. Nonetheless, in our case:

- The error rate (WER) level in the ASR process is quite low – 3.2%. This suggests that the mistakes in recognition (such as "a" instead of "the") are insignificant;
- In terms of BLEU score, the mistakes are mainly caused by the inability of the used models. Replacing Argos with MarianMT increases BLEU score by 48%;
- In regard to the prosodic performance, it is determined by the MMS-TTS engine itself. Ideal translation still appears to be robotic.

B. Argos Translate vs. MarianMT: The Latency-Quality Trade-off

The comparison of the Argos Translate engine (BLEU score: 15.3; latency: 0.4 s) and the MarianMT engine (BLEU score: 22.7; latency: 1.3 s) emphasizes the inherent latency-quality trade-off in real-time S2ST applications.

Why Argos Translate?

1) **Real-time constraint**: When translating streaming data, latency in translation should be under 0.5 s, otherwise, the system becomes sluggish, especially when coupled with a latency of 1.5 s for TTS.
2) **User tolerance**: Users tolerate inaccurate translation if latency is lower but are not happy with accurate but delayed (above 3 seconds) translation. This observation was made in a non-scientific experiment (unreported in this paper).
3) **Offline deployment**: Offline mode is achieved with Argos Translate thanks to packaged models. MarianMT (by Hugging Face Transformers) requires downloading its model each time when used.

When MarianMT?

MarianMT is preferable for:

- **Batch translation**: Since it is performed offline, the latency problem is less significant here, so a better BLEU score is a priority.
- **GPU deployment**: The latency of MarianMT is decreased to 0.3 s, when implemented on GPU hardware.
- **High-stakes scenarios**: Legal, medical, technical, or any other cases where accuracy is more valuable than speed.

Further research might involve the usage of a combination strategy: Argos Translate for real-time streaming and MarianMT for refining purposes.

C. TTS as the Latency Bottleneck

The MMS-TTS synthesis time of 1.5 seconds takes up 52% of processing latency and thus constitutes the primary bottleneck. This is due to three factors:

1) **Model size**: The Hindi MMS-TTS model has a total of 200M parameters, which is more than the sum of the ASR (74M) and the MT model (60M).
2) **Autoregressive decoding**: VITS-based TTS generation of waves is done through an autoregressive process, which requires sequential processing and does not allow parallelization..
3) **CPU inference**: The TTS models are designed for GPU inferences, making CPU inferencing 3–5x slower.

Mitigation Strategies:

- **Faster TTS models**: The AI4Bharat IndicF5 claims a synthesis time of 0.5s for Indian languages, which is 3x faster than MMS-TTS. However, IndicF5 came after the prototype and needs testing.
- **Streaming TTS**: Currently, TTS generates waves for the whole sentence before starting to play audio. The streaming TTS generates waves progressively and helps lower latency because it starts to play the audio faster.
- **GPU acceleration**: Running on a moderately powerful GPU, such as NVIDIA GTX 1650, lowers the latency to ~0.4 seconds, which reduces overall latency below 2 seconds.

D. Context Window: Design Choices and Limitations

As expected, the rolling context window with N = 2 effectively decreases repetitions (7 → 1) and removes grammar errors (4 → 0). Nevertheless, there are a few aspects that need discussion:

1) N = 2: Why Not Less or More?

We experimentally verified that N ∈ {1, 2, 3, 5}:

- *N = 1: Not enough context, repetitions persist (5 instances).*
- *N = 2: Best trade-off, repetitions decrease to 1, but latency remains low.*
- *N = 3: Small improvement (0 repetitions) at the cost of 20% more latency.*
- *N = 5: No further improvement, latency rises by 50%.*

Two is the best value, since it allows to provide sufficient context for conversational exchanges (our target application), but doesn't increase latency too much.

2) Text-Level vs. Speech-Level Context Window: While other approaches store context on the speech level, we store translated text. That decision has both pros and cons:

- **Pros:** The model gets to see what it has generated before, thus maintaining consistency in terminology and style.
- **Cons:** If the model makes an error in chunk n, it will make another error in chunk n+1 based on context. This is rare since errors usually affect single words, not whole phrases.

Instead, we could use source text as our context. But it wouldn't help to prevent repetitions, since the model won't know what it has already translated.

3) Heuristics in Extraction: Our heuristic approach (extracting the full translation of the prefix) assumes that the model generates the same phrase when it occurs as a prefix vs. separately. This works for deterministic, greedy models. For stochastic sampling (temperature > 0), however, the model will produce different translations for the same prefix in different contexts. Furthermore, reordering languages might also cause issues, as the prefix is likely to be reordered when translated in context.

Luckily, English-Hindi (SVO with medium reorderings) is immune to this problem. Our future works will explore better approaches based on attention-based alignment, for example.

E. Limitations and Threats to Validity

1) Translation Quality: Argos Translate's BLEU score (15.3) is significantly lower than commercially available systems (usually in the 30-40 range and above). In informal inspection, typical problems include:

- **Gaps in vocabulary**: Rare words and proper names are frequently mistranslated or improperly transliterated.
- **Idiomatic expressions:** Phrases like "break a leg" are translated literally.
- **Gender agreement: I**n Hindi, the agreement between the noun and the adjective is obligatory, and Argos Translate often misses this point.

Such limitations make the model unusable in high-stake translations (like consultations with a doctor, for instance), though regular communication is still not affected by them..

2) Test Corpora: Instead of real-world conversational data, our testing includes artificial datasets such as LibriSpeech, 25-sentence MT set, and 10-sentence context test. Limitations:

• **LibriSpeech:** Audiobook read speech differs significantly from the spoken one in its prosody, disfluencies, and vocabulary.

• **Custom MT set:** 25 sentences are insufficient to include all possible linguistic phenomena; a standardized testing corpus (FLORES-200, for instance) provides a more complete picture.

• **Context test:** 10 sentences are too little for testing the long-range dependencies (changes in topics, dialog turns, and so on). In the scope of further research, we plan to evaluate the solution on the corpora with conversational data (e.g., benchmarks for IWSLT simultaneous translation [16]).

3) Only One Translation Direction: We tested the EN-HI pair of languages, while expanding the set may reveal some other limitations:

- **Morphology**: Agglutinative languages (such as Tamil, Telugu: The use of a language which is written in another.
- **Script:** (Devanagari, Bengali) may allow identifying pronunciation errors in TTS synthesis.
- **Resource limitation:** Argos Translate covers 40+ languages, though there are some differences in quality among models; some pairs (for example, English-Marathi) have BLEU < 10.

4) No Voice Activity Detection: Due to processing in fixed-length chunks, the solution suffers from a number of disadvantages, including:

- **Mid-word interruption:** When speech goes beyond the length of the chunk, words can be interrupted mid-air.
- **Unnecessary silence processing**: Chunks that contain only silence get processed needlessly.

Using voice activity detection (for example, Silero VAD, WebRTC VAD) could segment speech into naturally ending chunks, thus improving both performance and efficiency. This would add latency, however, which we left for future work (50-100ms).

F. Generalizability to Other Use Cases

Although our application, PolyglotTalk, focuses on multilingual communication in India, the architecture is general enough for other scenarios with limited connectivity:

Humanitarian aid: Allows aid workers to communicate with refugees in low-connectivity environments.

Education: Enables translation in multilingual schools in areas where Internet access is unreliable.

Healthcare: Translates communications between doctors and patients in rural clinics with limited infrastructure.

Privacy-sensitive use cases: Translation is done on-device to ensure confidentiality (for example, legal consultation, therapy session).

Our solution's context window is not language-specific and can be applied to any cascade-based S2ST.

VII. Future Work

Several pipeline-level enhancements are left for future work. Integrating a lightweight Voice Activity Detection module — such as Silero VAD — would replace fixed-interval audio chunking with speech-boundary-aware segmentation, eliminating the incomplete-word and silent-gap artefacts inherent in the current overlap-based approach. Model quality could be substantially improved by substituting the present general-purpose components with Indic-specialised alternatives: IndicConformer for ASR (demonstrating 20–30% WER reduction over Whisper on Indian English and code-mixed speech), IndicTrans2 for machine translation (~30–40 BLEU on Indic language pairs versus Argos Translate's current baseline), and IndicF5 for TTS synthesis (projected to reduce end-to-end latency by approximately 35% over MMS-TTS). Each substitution comes with its own set of trade-offs in terms of model size and computation costs, both of which need careful benchmarking before being used in any production setting.

On the systems front, scaling up PolyglotTalk to support all 22 scheduled languages in India entails solving the challenges of automatic detection of the language and proper routing of the query to the relevant models, as well as handling cases where certain Indic-Indic pairs cannot be translated using the current libraries for machine translation. In the long run, the plan would be to develop a mobile application that will provide a two-way real-time translation facility using voice calls, thus necessitating further work on quantisation and optimised ARM-based inference. The complete codebase, benchmarking scripts, and evaluation data will be released open-source to support reproducibility and community extension of this work.

VIII. Conclusion

PolyglotTalk showcases the viability of an offline S2ST model built using cascade learning optimized for CPU-based processing. By trading the complexity associated with end-to-end approaches for modularity with a pipeline approach, which involves fast-whisper ASR, Argos Translate MT, and MMS-TTS TTS models, PolyglotTalk ensures under 3 seconds delay in translating audio on commercial hardware. The key innovation of this project is a rolling window context mechanism that retains grammaticality and semantics without having to resort to re-training. Although the system has shortcomings such as low BLEU scores and no VAD capability, it sets the groundwork for future development of communication platforms accessible even in poor Internet environments. Upcoming work would involve integration with Indic language specific models such as IndicTrans2, and IndicF5, among others.

References

[1] Google Research Blog, "Real-Time Speech-to-Speech Translation," November 2025. [Online]. Available: https://research.google/blog/

[2] Y. Jia, R. J. Weiss, F. Biadsy, W. Macherey, M. Johnson, Z. Chen, and Y. Wu, "Direct speech-to-speech translation with a sequence-to-sequence model," in *Proc. Interspeech*, 2019, pp. 1123–1127.

[3] Y. Jia, M. Tadmor Ramanovich, T. Remez, and R. Pomerantz, "Translatotron 2: High-quality direct speech-to-speech translation with voice preservation," in *Proc. ICML*, 2021, pp. 4901–4912.

[4] A. Agranovich, E. Nachmani, O. Rybakov, Y. Ding, and Y. Jia, "SimulTron: On-device simultaneous speech to speech translation," *arXiv preprint arXiv:2406.02133*, 2024. DOI: 10.48550/arxiv.2406.02133

[5] Q. Fang, Y. Zhou, and Y. Feng, "DASpeech: Directed acyclic transformer for fast and high-quality speech-to-speech translation," *arXiv preprint arXiv:2310.07403*, 2023. DOI: 10.48550/arxiv.2310.07403

[6] L. Barrault, Y.-A. Chung, M. C. Meglioli, D. Dale, *et al.*, "Seamless: Multilingual expressive and streaming speech translation," *arXiv preprint arXiv:2312.05187*, 2023. DOI: 10.48550/arxiv.2312.05187

[7] H. Inaguma, S. Popuri, I. R. Kulikov, P. Chen, and C. Wang, "UnitY: Two-pass direct speech-to-speech translation with discrete units," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[8] V. Pratap, A. Tjandra, B. Shi, P. Tomasello, A. Babu, S. Kundu, A. Elkahky, Z. Ni, A. Vyas, M. Fazel-Zarandi, A. Baevski, Y. Adi, X. Zhang, W.-N. Hsu, A. Conneau, and M. Auli, "Scaling speech technol-ogy to 1,000+ languages," *Journal of Machine Learning Research*, vol. 24, no. 97, pp. 1–52, 2023.

[9] V. Pratap, A. Tjandra, B. Shi, P. Tomasello, A. Babu, S. Kundu, A. Elkahky, Z. Ni, A. Vyas, M. Fazel-Zarandi, A. Baevski, Y. Adi, X. Zhang, W.-N. Hsu, A. Conneau, and M. Auli, "Scaling speech technol-ogy to 1,000+ languages," *Journal of Machine Learning Research*, vol. 24, no. 97, pp. 1–52, 2023.

[10] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," 2023. [Online]. Available: https://github.com/SYSTRAN/faster- whisper.

[11] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv preprint arXiv:2212.04356*, 2022.

[12] P. J. Finlay, "Argos Translate: Open-source offline translation library," 2020. [Online]. Available: https://github.com/argosopentech/argos-translate.

[13] J. Tiedemann, "Parallel data, tools and interfaces in OPUS," in *Proc. LREC*, 2012, pp. 2214–2218.

[14] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proc. ICASSP*, 2015, pp. 5206–5210.

[15] M. Post, "A call for clarity in reporting BLEU scores," in *Proc. Third Conference on Machine Translation (WMT)*, 2018, pp. 186–191.

[16] M. Morris, "jiwer: Similarity measures for automatic speech recognition evaluation," 2020. [Online]. Available: https://github.com/jitsi/jiwer.

[17] IWSLT, "International Conference on Spoken Language Translation: Simultaneous Translation Shared Task," 2024. [Online]. Available: https://iwslt.org/2024/.

[18] M. Ma, L. Huang, H. Xiong, R. Zheng, K. Liu, B. Zheng, C. Zhang, Z. He, H. Liu, X. Li, H. Wu, and H. Wang, "STACL: Simultaneous translation with implicit anticipation and controllable latency using prefix-to-prefix framework," in *Proc. ACL*, 2019, pp. 3025–3036.

[19] A. Polak, Y. Adi, J. Copet, E. Kharitonov, A. Lakhotia, W.-N. Hsu, A. Mohamed, and E. Dupoux, "SpeechMatrix: A large-scale mined corpus of multilingual speech-to-speech translations," in *Proc. Interspeech*, 2022, pp. 2993–2997.

[20] AI4Bharat, "IndicConformer: Speech recognition mod- els for Indian languages," 2023. [Online]. Available: https://github.com/AI4Bharat/IndicConformer.

[21] AI4Bharat, "IndicF5: Text-to-speech models for Indian languages," IIT Madras, 2024. [Online]. Available: https://github.com/AI4Bharat/Indic- F5-TTS.